卒　業　論　文

eBPF を用いた低遅延温度モニタリング基盤の設計と実装

22TC065　**對馬 秀悟**

指導教員　**宮澤 元**

2026 年 1 月

南山大学 理工学部 電子情報工学科

Design and Implementation of a Low-Latency Temperature Monitoring Platform Using eBPF

2022TC065　Tsushima Shugo

Supervisor　Miyazawa Hajime

January 2026

Department of Electronics and Communication Technology
Faculty of Science and Engineering
Nanzan University

要約

ここに本論文の要約を書く。要約は論文のエッセンスを抜き出したものであるので、ここで取り扱う問題、その問題を解決するための手法、および主な成果が書かれていなければならない。要約を読むだけで、論文の概要が分かり、読者にとって興味を抱く内容か否かが分かるようになっている必要がある。

日本語と英語の両言語で要約を書くが、必ずしも 1 対 1 に対応する文章になっている必要はない。それぞれにふさわしい表現があるからである。

Abstract

In this part, the abstract of this paper is described. Since 'abstract' means the essence or summary of the paper, the abstract should include the description on the problems treated in the paper, the author's approach for solving those problems, and the main results. Readers will understand the outline of the paper, without reading other parts, and be able to decide whether they will have interests in the paper or not.

目次

第 1 章　はじめに 1

1.1 概要 1
1.2 目的 1
1.3 貢献 1
1.4 構成 1

第 2 章　背景 3

2.1 エッジデバイスの普及と現状 3
2.2 エッジデバイスの発熱 3
2.3 発熱による不都合 3
2.4 エッジデバイスにおける冷却 3
2.5 冷却手段の課題点 4
2.6 アプローチ 4

第 3 章　設計 5

3.1 システムの概要 5
3.2 温度監視 5
3.3 タスクの分類 5
3.4 温度抑制スケジューリング 6
3.5 評価指標 6

第 4 章　実装 7

4.1 実装の概要 7
4.2 優先度制御 8
4.3 タスクの滞留 8
4.4 温度抑制 9

第 5 章　実験結果 11

5.1 実験環境 11
5.2 実行時間と安定性 11
5.3 温度変化 13
5.4 タイムスライス長と公平性 14
5.5 優先する割合と実行時間の関係 14
5.6 飽和状態における挙動 16
5.7 まとめ 17

第 6 章　考察 19

6.1 温度認識型スケジューリングの有効性 19

6.2 タスク実行の公平性と応答性のトレードオフ 19
6.3 優先タスク割合の閾値 19
6.4 タイムスライス長とオーバーヘッドの評価 20
6.5 sched_ext および eBPF の導入効果 20
6.6 既存の冷却手法との比較 20
6.7 高負荷・飽和状態における限界と課題 21

第 7 章 おわりに 22
7.1 研究全体の総括 22
7.2 本研究で得られた知見 22
7.3 本研究の課題 22
7.4 今後の展望 23

参考文献 24

第 1 章

はじめに

本章では, 本研究の立ち位置と目的を明確にするための導入として研究の全体像について述べる. まず 1.1 節研究の概要を説明する.1.2 節で研究の目的を,1.3 節では本研究の貢献点を,1.4 節では本論文の構成を示す.

1.1 概要

近年,IoT の普及に伴いエッジコンピューティングの重要性が急速に高まっている. Raspberry Pi 5 などのエッジ端末は監視カメラの画像処理やセンサーデータのリアルタイム解析など, 高度な演算処理を実行するために広く利用されている. しかし, それらの機器は冷却機能が簡易的であることが多く, 高負荷時には CPU 温度が上昇することによるサーマルスロットリングが発生し, システム全体の処理能力が著しく低下するという課題がある. 本研究では,Linux カーネルの拡張機能である sched_ext を用いてタスクの重要度と CPU 温度に基づいた動的なスケジューリング手法を実装し, 高負荷時においても重要なタスクの応答性を維持しつつ, 発熱を抑制可能なシステムを提案する.

1.2 目的

本研究の目的は, タスクの実行頻度を重要度に基づいて動的に変更することで, エッジデバイスにおける温度抑制と, システム全体の最低限のスループット維持の両立を実証することである.

評価においては, 作成したスケジューラとデフォルトのスケジューラでそれぞれ同様の負荷をかけ, スループット, 応答性, および CPU 温度の推移を比較検証する.

1.3 貢献

本研究の貢献は以下の点である.

- Linux 6.12 で導入された sched_ext を用いて, ユーザー空間で定義可能な温度認識型スケジューラを設計・実装した点.
- タスクを優先するグループとそうでないグループに分類し, 温度上昇に伴って実行比率を動的に変化させるアルゴリズムを確立した点.

1.4 構成

本論文の構成は以下の通りである.

第 2 章ではエッジコンピューティングにおける熱問題と既存の冷却手法, およびその課題について述べる.

第 3 章では本研究で提案する温度・重要度に基づくスケジューリング方式の設計について述べる.

第 4 章では sched_ext を用いた実装の詳細, 特にキュー深度を用いた負荷判定と温度ステージ制御について説明する.
第 5 章では実験結果を示す.
第 6 章では実行時間と温度変化の観点から提案手法の有効性を評価・考察する.
第 7 章では本研究のまとめと今後の課題について述べる.

第 2 章

背景

本章では, 本研究の背景となるエッジデバイスの熱問題とその解決策について述べる. まず 2.1 節でエッジコンピューティングの普及と現状を説明する.2.2 節および 2.3 節でデバイスの発熱メカニズムとその弊害を,2.4 節では既存の冷却手法について概説する. 最後に 2.5 節で既存手法の課題を指摘し,2.6 節で本研究のアプローチを示す.

2.1 エッジデバイスの普及と現状

近年, データ処理を中央のクラウドではなく, データの発生源近くで行うエッジコンピューティングが注目を集めている. 従来のクラウド中心のシステムでは高性能なサーバで処理できる代わりに膨大な量のデータを遠隔地に送信する必要があったり通信におけるセキュリティが問題視されていた. しかし, エッジデバイスの処理能力の向上に伴い, 従来はクラウド側でしてきた処理をエッジが担う機会が増えてきた. これにより, リアルタイム性, プライバシー保護の両方を担保することが可能になった.

2.2 エッジデバイスの発熱

エッジデバイスの処理能力の向上に伴い, 発熱量も増加している. 大規模な演算を実行する過程で大量の電力を消費し, 電力による熱生成が発生するためである. 集積度が高いデバイスや小型なデバイスほど単位面積あたりの熱発生量が増大し, 発熱が顕著になる.

2.3 発熱による不都合

エッジデバイスの発熱は単なる温度上昇にとどまらず, システムの性能やハードウェアの劣化を引き起こす. 具体的には

1. 強制的に CPU 周波数の上限を落とすことで消費電力そのものを抑える
2. ハードウェアの劣化温度が 10 ℃上昇すると故障率が倍になるアレニウスの法則

高い周波数を維持することはそれだけ早くタスクを完了させられることでもあるが, 同時にエッジデバイスの寿命を削る行為にもなり得る. この様に温度上昇は信頼性や安全性の面で無視できないリスクを伴う.

2.4 エッジデバイスにおける冷却

エッジデバイスで主流な冷却方法はハードウェア, ソフトウェアの両方で行うことができる.
ハードウェア的手法ではヒートシンクなどの高温側から低温側に熱が直接伝得る方式とファンなどの空気が動くことで熱を外部へと運ぶ方式の 2 種類がある.

ソフトウェア的手法だと温度抑制の手法として DVFS(Dynamic Voltage and Frequency Scaling) を用いたものが標準的である. これは CPU 温度が一定の値を超えると CPU 周波数を強制的に低下させることで発熱を抑えるというものである.

また,MARTIN らは模倣学習と AI 処理に特化したプロセッサーを併用することで温度を最大約 17 ℃下げることに成功している [1].

2.5 冷却手段の課題点

様々な冷却機構が導入されているが, 技術的な限界や使用環境によって導入できない方法なども存在する.
例えば, 静音性や塵, 埃などの観点からファンを持たない機器が求められたり, ヒートシンクなどもサイズやコスト, 高出力熱源に対応しきれないなどの問題がある. また,DVFS はシステム全体の性能を一律で抑制することになるため, ユーザーにとって大切なタスクの処理も遅延してしまうことになる. エッジデバイスに求められるリアルタイム性が損なわれたり, そもそもこういったスロットリングはかなりの高温時における緊急的な意味合いが強いので, 先述の信頼性や安全性を損なってしまう.

2.6 アプローチ

したがって, エッジデバイスの長期信頼性を確保するためには平均温度を低く保つようなスケジューリング制御が必須となる.
以上の課題に対し, 本研究では温度を下げるために全タスクを一律に遅くするのではなく, タスクの重要度に応じて実行頻度を調整することで重要なタスクの応答を守りながら温度上昇を抑えることを試みる.

第 3 章

設計

本章では本研究で提案するシステムの全体設計について述べる. 本研究は負荷が大きい処理を実行している環境を前提とし, そのような環境下であっても温度抑制と優先したい処理の応答性の両立を目指すものである.

3.1 節ではシステムの概要について,3.2 節では温度監視について述べる.3.3 節ではタスクの分類方法を,3.4 節では温度抑制スケジューリングについて,3.5 節では評価指標について述べる.

3.1 システムの概要

本研究では負荷抑制にあたりコンピュータが処理する仕事の単位であるタスクに着目する. タスクの重要度に基づいて実行頻度を変えることで特定のタスクの実行頻度を守る. 以下の構成で設計・実装する.

- 現在の温度を確認する
- 現在の温度に基づき, 優先度の高いタスクの実行頻度を高くする

3.2 温度監視

提案する温度監視システムについて述べる. 温度は 3 つのステージで扱う. 一定間隔で温度を取得し, 必要であればステージを更新する. 状態遷移にはヒステリシスを設け, 頻繁な状態の変動を防止する.

COOL　温度が低い状態. 優先タスクもその他も同じ割合実行する
WARM　温度が上昇傾向にあるので, 若干優先タスクの重みを下げる
HOT　強力な制限をかける.

3.3 タスクの分類

以下表にタスクの種類を示す.

タスクの種類	性質	制御内容
カーネルタスク	OS の機能など, システムの維持に不可欠な処理	制御対象外
優先タスク	締切が近いタスク又はユーザーにとって価値の高いタスク	温度上昇に伴い, 優先的に処理される
その他タスク	重要度の低い処理	温度上昇に伴い, 制限される

3.4 温度抑制スケジューリング

CPU に実行すべきタスクがなく, 暇にしている状態をアイドル状態という. この間,CPU は少電力や発熱の抑制に努めている. そこで温度ステージが上昇するにつれて CPU がアイドル状態になるようにその他タスクの実行頻度を減らす様に設計する.

3.5 評価指標

本研究では作成したプログラム実行時とデフォルトの状態でそれぞれ同じ負荷をかけ, かかった時間と温度推移を評価する.

負荷をかけるにあたって, 効率よく負荷をかけられる浮動小数点演算を採用した. 100 個の負荷を子プロセスで展開して, 全て実行可能になったことを確認してから一斉に実行する. 実行してから全ての子プロセスが終了するまでの時間を計測する.

第 4 章

実装

本章ではシステムの実装の内容や実験手順について述べる.

4.1 節では実装の概要を 4.2 節では優先度による制御について,4.3 節ではタスク残留時の挙動を,4.4 節では温度抑制の実装方法について述べる.

4.1 実装の概要

4.1.1 システムの基盤

システム基盤には Linux カーネルの機能を安全かつ動的に拡張するための技術である eBPF を採用した. ユーザー空間で作成したプログラムを厳格に検証した後にカーネル内に差し込むことで機能を拡張できる [2]. カーネル空間での実装のため, オーバーヘッドを抑えつつ高度な機能を追加できる.

4.1.2 スケジューリングの実装

本研究では実装にあたり,Linux カーネルのスケジューラ拡張機能である sched_ext を採用した. これはユーザー空間から動的にスケジューリングポリシーを定義, 切替可能にするフレームワークである [3].sched_ext を採用した理由は安全性や実験の容易さなどがある. 具体的には

- タスクの管理や実行順序を体系的に設計できる
- 新たなスケジューリングポリシーをカーネルの再構築無しで行える
- 検証機能がついているので, ロードするポリシーがシステムを破壊しないことが保証される
- 万が一スケジューラがエラーを起こした場合は自動でデフォルトの状態に切り替わる

などがある.

4.1.3 温度監視

温度監視は低いオーバーヘッドで CPU の温度をはかることができる ebpf のフックポイントである tracepoint/thermal/thremal_temperature を常時参照する.

4.1.4 タスクの分類

本システムの実装はタスクの特性に応じたスケジューリングを実装するため,SCX が提供する Dispatch Queue(以後 DSQ) を使用する. これはタスクを入れるキューであり, 各 CPU に 1 つずつあるローカル DSQ, ユーザーが任意個作成できるカスタム DSQ がある. また, ローカル DSQ は FIFO で, カスタム DSQ は FIFO 及び優先度キューのどちらかで運用する.

タスクはカーネル, 優先, その他に分類するようにしたので, それぞれの名前は以後 KERNEL,FAST,NORMAL とする.
カーネルタスクと優先タスクはどちらも応答性を担保したいタスクであるが, 分けた理由はカーネルタスクはそれ以外のタスクよりも桁違いに多い上にわずかでも遅れればシステム全体の遅延を招く恐れがあるからである.

4.1.5 動作原理

以下にタスクがどのような流れでカスタム DSQ に挿入され, どのような流れでローカル DSQ に挿入されるのかのフローとタスクが実際に処理される際のフローを示す.

CPU はまず自分のローカル DSQ からタスクを取得する. ない場合はカスタム DSQ からタスクを取得する.

ローカル DSQ に振り分けられるフロー

1. タスクが実行可能状態になる
2. どのコアで処理するかの決定
3. どのカスタム DSQ へ入れるかを決定
   (a) カーネル用のタスクであれば KERNEL へ移動
   (b) それ以外で指定した要件を満たす場合もしくは宣言されている場合は FAST へ移動
   (c) それ以外は全て NORMAL へ移動

タスクが処理されるフロー

1. CPU がタスクを欲する
   (a) 現在の温度ステージに基づき処理させるタスク量を決定
   (b) カーネルタスクは無条件でローカル DSQ へ移動
   (c) その他のタスクは各群から決められた分だけ先程のフローで決定した CPU のローカル DSQ へ移動

なお,FAST に挿入されるタスクは締切が近いタスクと指定した PID のタスクの 2 種類がある. これは本研究ではどの程度の割合が優先的に処理されるのが最も効率が良いかを測る意図がある.

また, 優先するタスクは全体のおよそ 1 割程度を見込んでいる. 全体の数値の大部分は一部の要素から生み出されるというパレートの法則があるが, そこでは大部分と一部の要素の比は 8:2 となっている. これに基づき, 優先すべきタスクの割合は全体の 1 割とした.

4.2 優先度制御

タスクの実行順序決定には, クレジット方式を採用した. 各 CPU ごとに FAST 用と NORMAL 用のクレジットを管理し, 温度ステージに応じて定義された比率でクレジットを消費しながらタスクを CPU に渡す.

例えば, 高温状態では FAST 側のクレジット配分を大きく,NORMAL 側の配分を小さく設定することで,NORMAL タスクの実行頻度を物理的に低下させている.

4.3 タスクの滞留

4.3.1 ウォッチドック機能

高温時は NORMAL から取り出す量を減らす様に設計したが, 高温状態が続くと NORMAL にタスクがたまり続けていくことになる. キューにタスクが長時間たまり続けると実行可能状態にもかかわらず実行されない状態が発生してしまう. この問題に対処するため,sched_ext にはウォッチドック機能という全ての実行可能

状態のタスクを監視し, 一定秒数実行されない場合は自動的にカーネル標準のスケジューラーにリカバリーするというものが実装されており, この仕組みによってシステム全体の安全性と応答性が担保されている.

4.3.2 対策

本実装では, 単に温度のみで制御を行うのではなく, タスクの滞留状況を考慮するためにキュー深度を用いた制御を導入している. 制御するのは

- FAST に割り当てる条件
- CPU の持ち時間
- CPU のタイムスライス

の 3 点となっている.
sched_ext フレームワークにおいては, 各カスタム DSQ に待機しているタスク数を scx_bpf_dsq_nr_queued() 関数を用いて取得可能である. 本システムでは, ディスパッチのタイミングごとに各カスタム DSQ の合計タスク数を計測し, これを現在のシステム負荷として定義している.

タスクが大量に溜まっている場合, システムは FAST の判定基準を厳しくし, より緊急度の高いタスクのみを FAST として扱うように振る舞う. これにより, オーバーロード時においてもシステムが破綻することなく, 重要な処理のリソースを確保する仕組みとなっている.
また, カスタム DSQ に溜まっているタスク数が閾値を超えた場合は個別のタスクの応答性ではなく, 全体の応答性に着目する必要が出てくる. そのため,NORMAL キューにタスクが溜まっている場合はデフォルトスケジューラのように公平性を重視するため,CPU のタイムスライスを短くするようにしている.

4.4 温度抑制

温度抑制をどの様に行っているかを以下図で示す. 高温時は優先タスクの実行割合を増やす. ただ, 先述のと

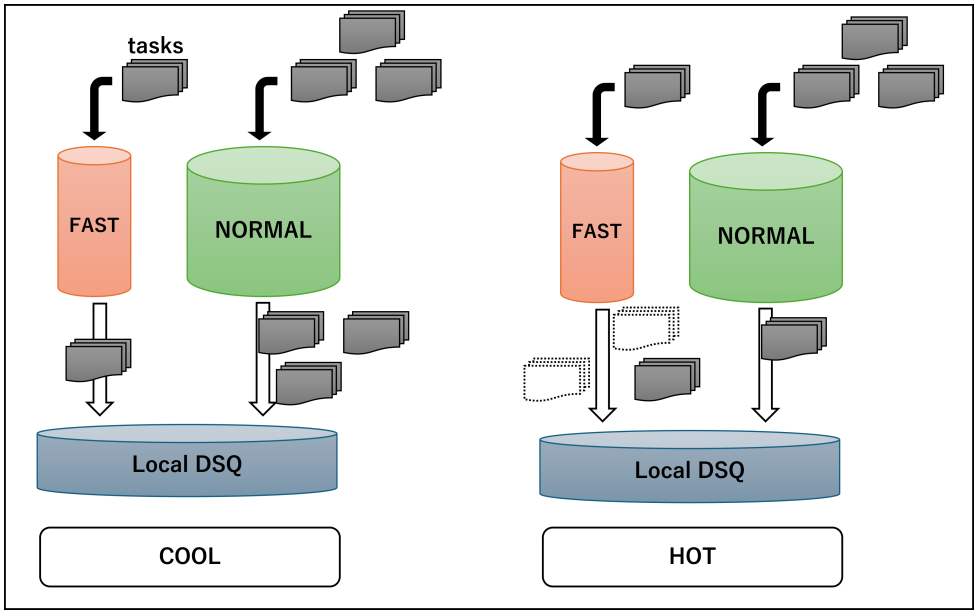

図 4.1 温度抑制

おり, 優先タスクの割合は 1 割程度を見込んでいる. そのため, 実行割合が増えても優先タスクが空である可能

性がある. その場合はタスクを待つことも NORMAL から変わりに取り出すこともなく CPU はアイドル状態となり, それが温度抑制につながる.

第 5 章

実験結果

5.1 節では実験環境について,5.2 節では実行にかかった時間と安定性について,5.3 節では温度推移を,5.4 節では CPU のタイムスライスと公平性について述べる.5.5 節では優先する割合の変化と実行時間の関係を,5.6 節ではタスク飽和時の挙動を,5.7 節では実験のまとめについて述べる.

5.1 実験環境

実験に使用したハードウェア及びソフトウェア環境を以下表に示す. ハードウェアには高い CPU の性能と

表 5.1 実験環境

使用機器	Raspberry Pi 5 Model(メモリ 8GB)
バージョン	Linux raspberrypi 6.12.34-v8-16k
SCX バージョン	v1.014-495-g8d646274

十分なメモリ容量を持つ 8GB の Raspberry Pi 5 を用いた.

sched_ext は Linux6.12 以降で導入されたものであるため,Raspberry Pi 5 のバージョンは 6.12.34-v8-16k を,sched_ext はそれに対応したバージョンを採用した.

5.2 実行時間と安定性

5.2.1 実験の目的

デフォルトのスケジューラでは公平性を担保するためにタイムスライスが設定されている. これはタスクを順々に処理するのではなく, 複数のタスクを並行処理するために用いられている手段である.
本研究では, 高い優先度のタスクの応答性を確保するために sched_ext のマクロで設定されている値である 20ms に設定した.
切り替えの回数が減り 1 つのタスクにかける時間が増えた分, 公平性を失ってしまっていると考えられる. そのため, 一度に大量の負荷をかけてその実行時間を箱ひげ図で見ることで公平性を観察する.

5.2.2 実験方法

100 個の子プロセスを起動し, そのうち 1 割の PID を FAST に登録した. 各子プロセスごとにかかった時間を計測した.

5.2.3　実験結果

デフォルト時とプログラム実行時の各温度ステージの実行時間は以下の箱ひげ図のようになった.

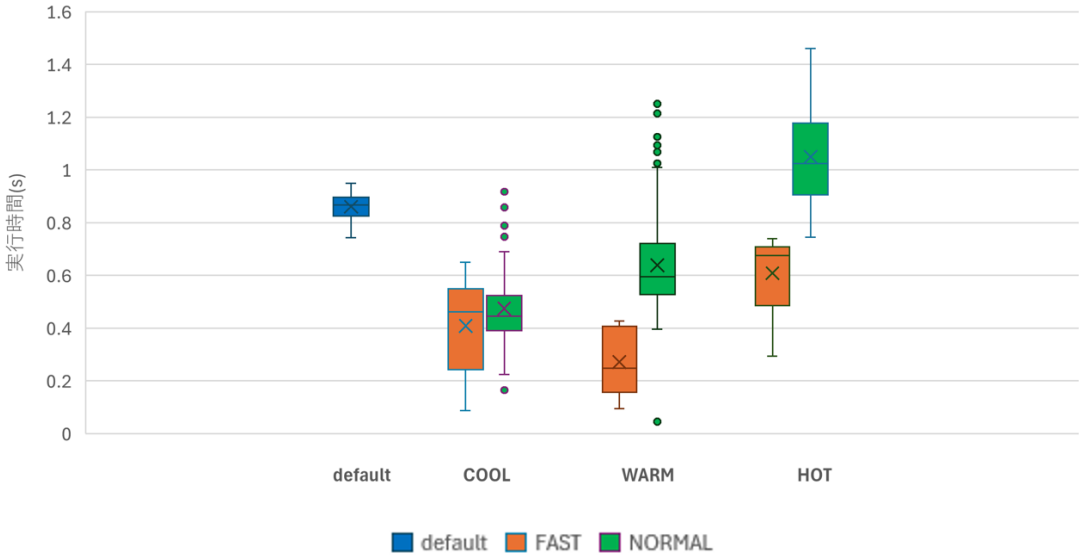

図 5.1　箱ひげ図

デフォルトのスケジューラは, 四分位範囲が狭く, ひげも短いため, タスクごとの実行時間のばらつきが非常に小さい. これは CFS や EEVDF といった標準スケジューラが公平性を重視し, 全タスクに対して均等に CPU 時間を割り当てている結果である.

対して, 提案手法では, 温度ステージが上昇するにつれて,NORMAL タスクの実行時間が大幅に伸長し, ばらつきも増大していることがわかる. 一方で,FAST タスクの実行時間は HOT 状態であっても比較的低く抑えられている.

これは, 温度上昇に伴い NORMAL タスクの実行権が制限され, 待機時間が増加したことによるものである. 重要なタスクの応答性能を維持するために, 重要度の低いタスクが犠牲となり, スループットが意図的に抑制されたことを示しており, 設計通りの挙動が確認できた.

5.3　温度変化

5.3.1　実験の目的

一時的に負荷が跳ね上がった場合の挙動は先程のセクションで示したが, 短時間の処理であるため温度上昇は見込めない. そのため, 温度が上がったときのステージ変化に伴う負荷対策の有効性を確かめるために行う.

5.3.2　実験方法

2 つの大きさが違う負荷を stress-ng コマンドで 15 分間に渡って負荷をかけ, 温度の推移を観察した.1 つは単一の大きな負荷を 15 分間, もう一方は小さめの負荷をかけ, それを繰り返し続けた. 小さめの負荷の方を優先タスクに登録した.

5.3.3　実験結果

作成したプログラムを動かしている時とそうでないときで同じ負荷をかけ, 温度変化を監視した. 以下にその結果を示す.
また, 以下表に平均値, 中央値, 最高値を示す.

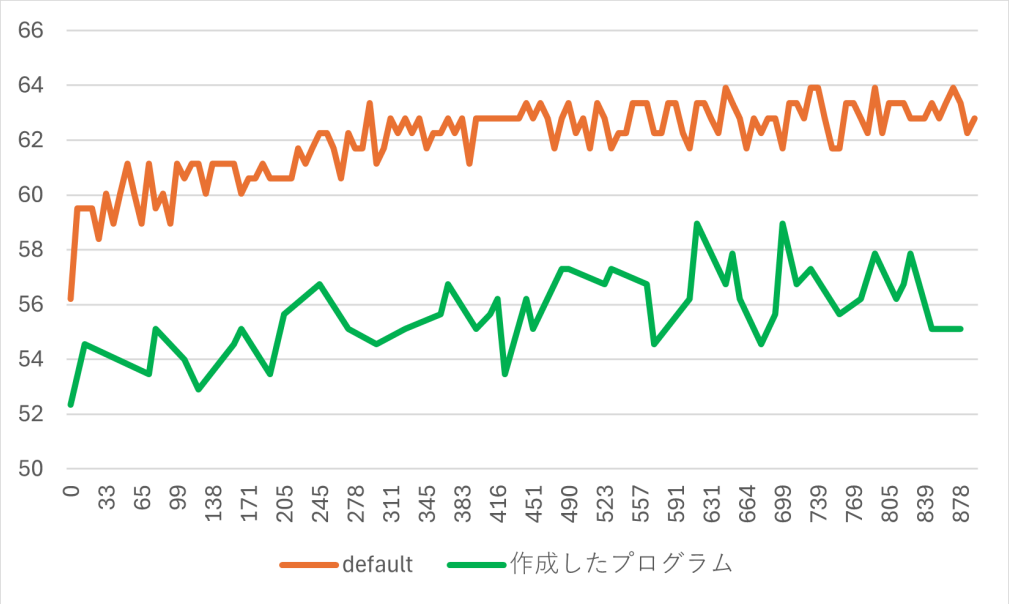

図 5.2　温度推移

表 5.2　実験結果

	デフォルト時	プログラム実行時
平均温度	62.02	55.78
中央値	62.25	55.65
最高温度	63.9	58.95

デフォルト状態では実験開始直後から急激に温度が上昇し, 約 15 分間の計測において平均 62.02 ℃, 最高 63.9 ℃に達している. これは CPU が常にフル稼働し, 熱飽和に近い状態にあることを示唆している.

一方, 提案プログラム適用時は, 平均 55.78 ℃, 最高 58.95 ℃と, デフォルト時と比較して平均温度で約 6.2 ℃の低下が見られた. グラフの推移を見ると, 温度上昇に伴うステージ変更により冷却制御が働き, 温度が一定範囲内で上下する推移を示していることがわかる.

これは, 温度閾値を超えた際に NORMAL タスクの間引きが行われ,CPU 負荷が下がって温度が低下し, その後再びタスク処理が再開されるというフィードバックループが正常に機能していることを裏付けている.

5.4 タイムスライス長と公平性

5.4.1 実験の目的

5.1 節で本研究では長めのタイムスライスを用いていると解説した. 切り替えが減ったことで実行完了時間は早くなったが, ばらつきが激しい. そこで, 提案手法よりも短いタイムスライスで実験をしたらどうなるのかを検証する.

5.4.2 実験方法

タイムスライスの長さがシステムのスループットと公平性にどのような影響を与えるかを定量的に評価するため, 以下の 4 条件で比較実験を行った.

- Default: Linux 標準のタイムスライス
- 提案手法 (20ms):sched_ext のマクロを使用したもの
- 10ms: タイムスライスを 10ms に設定した高スループット設定.
- 5ms: タイムスライスを 5ms に短縮し, 公平性を重視した設定.

5.4.3 実験結果

以下図に同一負荷ワークロードにおける各条件の箱ひげ図を示す.

中央右の 10ms 設定では,FAST・NORMAL ともに箱ひげ図の四分位範囲が大きく広がっている. これは, 運良く先に処理されたタスクと, 列の後方で待たされたタスクとの間で, 完了時間に大きな格差が生じていることを示している.

一方, 図右側の 5ms 設定を見ると,FAST の箱の縦幅が縮小しており,Default に近い形状となっている. これは, 短い間隔でタスクを切り替えることで, タスク間の進捗が平準化され, 公平性が改善されたことを示している. NORMAL の方を見ると, かなりの遅延が発生しているがこれは,BPF プログラムによるスケジューリング処理やコンテキストスイッチの頻度が増加したことで, そのオーバーヘッドが計算処理を圧迫したためと考えられる.

5.5 優先する割合と実行時間の関係

5.5.1 実験の目的

本研究の提案手法では, システム全体の負荷が高い状態において, 特定の重要なタスク（FAST）の応答性を維持するために, その他タスク（NORMAL）の実行を制限している.

ここでの重要なパラメータは「全タスクのうち, どの程度の割合を FAST として扱うか」である.FAST に分類されるタスクが少なければ少ないほど, そのタスクは手厚く保護されるが, 逆に FAST の割合が増えすぎれば,CPU リソースの奪い合いが FAST 同士で発生し, 優先制御の効果が薄れることが予想される.

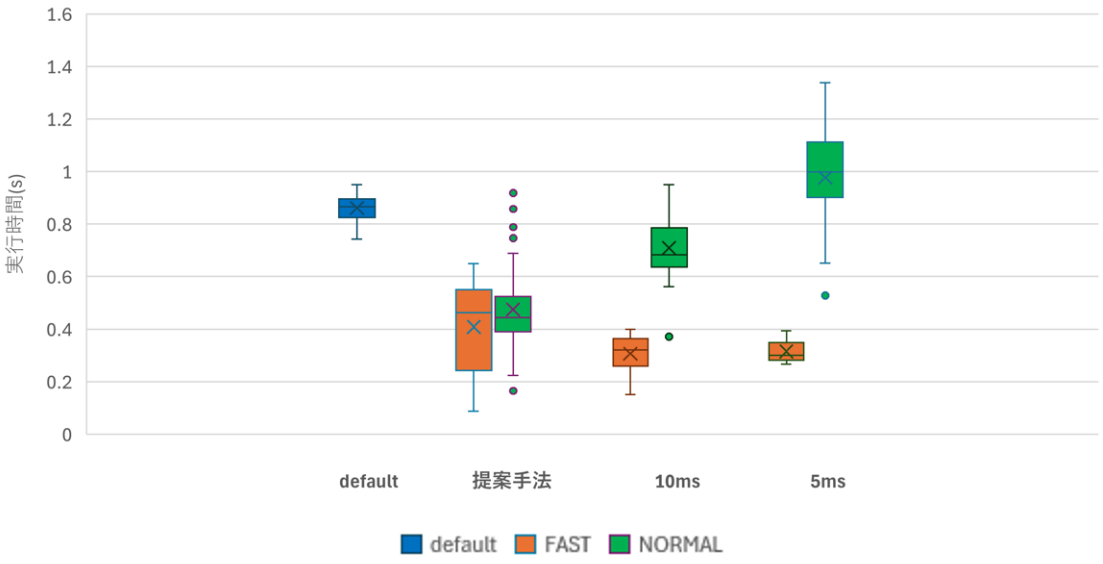

図 5.3

本節では,FAST に登録するタスクの割合を 0% から 100% まで変化させた際の実行時間の推移を測定し, 提案手法が有効に機能する適切な割合（閾値）を明らかにすることを目的とする.

5.5.2 実験方法

登録する割合を 0% から 20% ずつ増やしていき, 完了時間と温度推移がどの様に変化するのかを観察する. 合計 100 個の計算負荷プロセスを同時に起動し, そのうち FAST グループに登録するプロセスの割合を 0%, 10%,20%, 40%, 60%, 80%, 100% と段階的に変化させた.

それぞれの条件において,FAST タスク群と NORMAL タスク群の実行完了時間を計測し, その分布を比較した.

5.5.3 実験結果

実験結果より, 以下の傾向が確認された. 低割合（10%〜20%）における有効性:

提案手法（約 10%）および 20% の条件では,FAST タスクの実行時間が極めて短く抑えられている一方で,NORMAL タスクは待機させられていることがわかる. この範囲内であれば, 優先制御が極めて有効に機能していると言える.

中割合（40%〜60%）における性能劣化:

FAST の割合が 40% を超えると,FAST タスク自体の実行時間が顕著に増加し始めている. これは, 優先すべきタスクの絶対数が増えたため, 限られたクレジットを FAST タスク同士で分け合う形となり, 待ち時間が発生したためである.

高割合（80%〜100%）における破綻:

割合が 80% を超えると, もはや FAST と NORMAL の差はほとんど確認できなくなる. 特に 100% の条件では, 実行時間が 2.5 秒近くまで増大しており, これは 0% の条件よりも大幅に悪化している.

全タスクを優先しようとした結果, 頻繁なコンテキストスイッチやスケジューリングのオーバーヘッドが発生し, システム全体のスループットを低下させている.

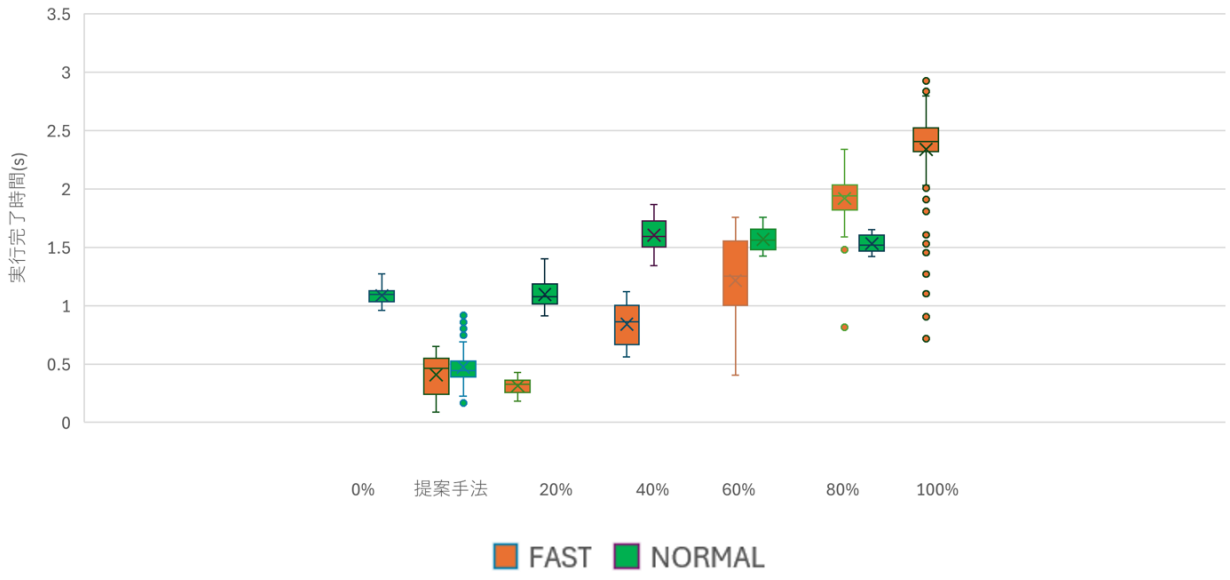

図 5.4

5.6 飽和状態における挙動

5.6.1 実験の目的

5.3 節ではアイドル状態を意図的に作り出すことで温度抑制が可能であることを示した. しかし, 処理すべきタスクが常に供給され続けるような極めて負荷が高い状況においては,CPU をアイドル状態にする隙間を作ることが困難となる.

本節では, アイドル時間の挿入が困難なほどの高負荷を与え続けた場合において, 提案手法がどのような挙動を示すかを検証する. 具体的には, 温度抑制効果の限界と, そのような極限状態における優先タスクの応答性が維持されるかを確認することを目的とする.

5.6.2 実験方法

前の実験よりも強度を高め, 常に CPU 使用率が 100% に張り付くような計算負荷を連続して投入し続けた.

評価データとして, 各タスクの実行時間と, 実験期間中の CPU 温度分布を取得した. 実行時間の比較対象は, デフォルト, 優先タスク, 通常タスクの 3 群である.

5.6.3 実験結果

以下図に温度の箱ひげ図を, 図に完了時間の箱ひげ図を, 表に温度の平均, 中央値, 最高値を示す.

表 5.3 実験結果

	デフォルト時	プログラム実行時
平均温度	73.8	71.83
中央値	75.45	74.35
最高温度	80.4	77.65

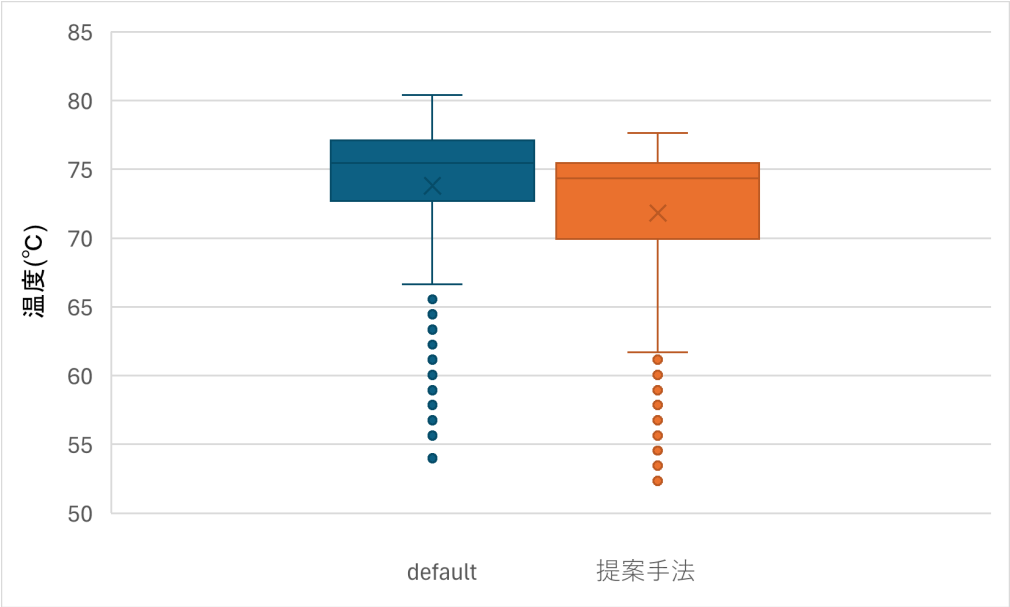

図 5.5

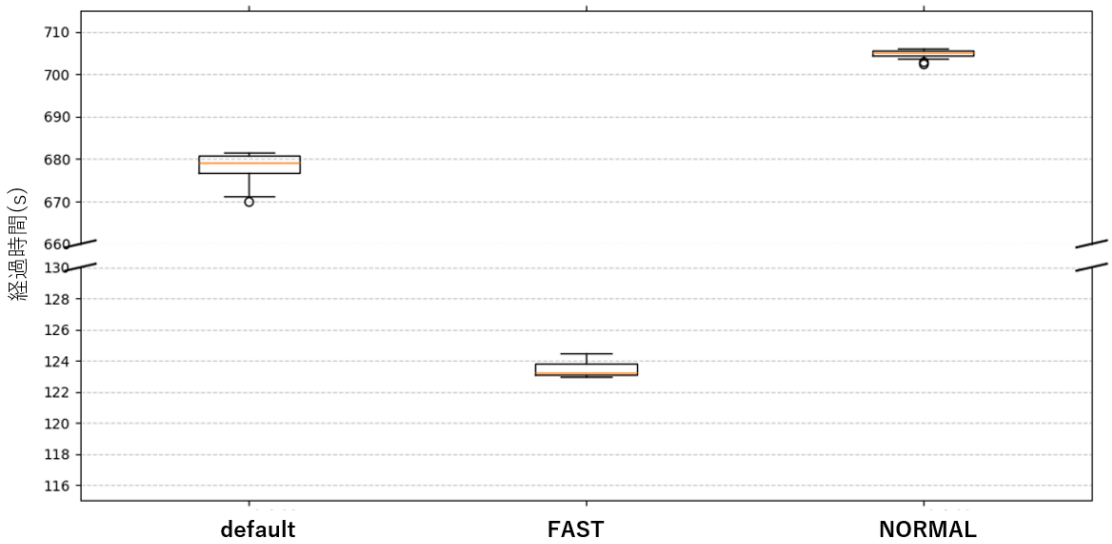

図 5.6

この結果から, システム全体が飽和するような高負荷状況下であっても, 提案手法の優先度制御は機能しており,FAST に分類されたタスクの応答性を改善できていることがわかる. NORMAL の応答性はデフォルトに比べて約 20 秒ほど遅くなっている.FAST 自体は開始 2 分ほどでこれはオーバーヘッドや実行比率現象による影響が出ているものと思われる.

5.7 まとめ

本章では, 提案手法である温度抑制スケジューリングの有効性を多角的に検証した.

まず, 実行時間と安定性の評価において, 提案手法は温度上昇時に通常タスクの実行を制限することで, 優先タスクの応答性を維持できることを確認した.

次に, 温度変化の測定では, デフォルトのスケジューラと比較して平均温度を約 6.2 ℃低下させることに成功し, 温度ステージ制御による冷却効果が実証された.

また, タイムスライスの比較実験により,20ms の設定はスループット向上に寄与するものの, 公平性が損なわれるトレードオフがあることが判明した.

さらに,FAST タスクの割合に関する実験では, 優先タスクが全体の 10% から 20% 程度であればシステムは有効に機能するが,40% を超えると優先制御の効果が薄れ,80% 以上ではシステム全体のスループットが悪化することが示された.

最後に,CPU 使用率が 100% に達する飽和状態においても, 提案手法は優先タスクの応答性を保護しつつ, 一定の温度抑制効果を発揮することを確認した.

これらの結果から, 本研究で実装したシステムは, 高負荷なエッジデバイス環境において, 温度抑制と重要タスクの実行保護を両立するという所期の目的を達成していると結論付けられる.

第 6 章

考察

本章では第 5 章で示した実験結果を基に提案システムの特性, 妥当性について詳細に考察を行う.

6.1 節では本システムの有効性について,6.2 節では公平性と応答性のトレードオフについて,6.3 節では優先タスクの割合と応答性が両立できる境界についてのべる.6.4 節では CPU のタイムスライス長とオーバーヘッドの関連性を,6.5 節では使用した技術の効果について,6.6 節では既存の手段との比較を行い,6.7 節では長時間の負荷による課題点について述べる.

6.1 温度認識型スケジューリングの有効性

提案手法を適用することで,Raspberry Pi 5 の高負荷時における平均温度を, デフォルトの状態と比較して約 6.2 ℃低下させることができた. これは,CPU 温度の上昇に伴い, 優先度の低いタスクの実行頻度を意図的に制限し,CPU にアイドル時間を発生させるという設計思想が, 熱エネルギーの総量抑制に寄与したことを示している.

特筆すべきは, 温度抑制を行いながらも, 優先タスクの実行時間がデフォルト時と同等, あるいはそれ以下の短い時間で維持されていた点である. これは, 温度上昇時においても重要なプロセスの応答性を損なうことなく, システム全体の熱管理が可能であることを実証している.

6.2 タスク実行の公平性と応答性のトレードオフ

提案手法適用時は NORMAL タスクの実行時間が大幅に伸長し, そのばらつきも増大していることが確認された. これは,FAST タスクの応答性を最優先した結果,NORMAL タスクに対する公平性が犠牲になったことを意味する.

一般的な汎用 OS のスケジューラは, 全タスクに対して CPU 時間を公平に分配することを重視する. しかし, 組み込みシステムや特定用途のエッジデバイスにおいては, 全てのタスクが同等の価値を持つわけではない. 本研究の結果は, 公平性を意図的に欠如させることで, 熱制約という物理的な限界内でのシステム稼働継続性を確保できることを示唆している.

6.3 優先タスク割合の閾値

全タスクに対する FAST タスクの割合を変化させた際,10% から 20% の範囲内であればシステムが有効に機能することが明らかになった. 一方で, その割合が 40% を超えると FAST タスク自身の実行遅延が顕著になり,80% 以上ではシステム全体のスループットが著しく悪化した.

優先されるべきタスクが増えすぎれば, 優先権そのものの価値が希薄化し, 限られた CPU リソースを奪い合うことになる. 本システムを実運用する上では,FAST として登録するタスクを厳選し, 全負荷の 2 割程度に留める運用設計が不可欠であるといえる.

6.4 タイムスライス長とオーバーヘッドの評価

タイムスライスを標準よりも長い 20ms に設定した提案手法と, 短い 5ms に設定した場合を比較した. 結果として,20ms 設定の方がスループットが高く,5ms 設定では公平性は向上するもののオーバーヘッドによる遅延が見られた.

sched_ext は eBPF を用いてユーザ空間でスケジューリングを行うため, カーネル空間内で完結する標準スケジューラと比較して, コンテキストスイッチやスケジューリング判定のコストが相対的に高い. そのため, タイムスライスを短く設定しすぎて切り替え頻度を上げると,eBPF プログラムの実行回数が増加し, それが CPU 負荷となって発熱要因になり得る. 本研究のような熱抑制を目的とする場合, あえてタイムスライスを長めに設定し, 切り替えコストを低減させる方針は合理的であったと評価できる.
しかしながら, 提案手法ではタイムスライスの変更を動的に設定していない. 現在のカスタム DSQ の容量だけではなく, 待ち時間などを考慮した上でタイムスライスを変更すれば高負荷な状況が続く場合であっても一定の応答性とさらなる温度抑制が可能になると思われる.

6.5 sched_ext および eBPF の導入効果

本研究では,Linux カーネル 6.12 で導入された sched_ext を採用した. 従来, 独自のスケジューリングポリシーを実装するにはカーネルのソースコードを直接改変し, 再ビルドする必要があったが, これは開発コストが高く, システムの安定性を損なうリスクも高かった.

これに対し,eBPF ベースの sched_ext を用いることで, 稼働中のシステムに対して動的にスケジューラをロード・アンロードすることが可能となり, 安全かつ迅速な実験が可能となった. さらに,eBPF の持つ軽量性は, 計算リソースの限られたエッジデバイスと極めて高い親和性を持つ. 通常, ユーザー空間で複雑な制御を行おうとすると, カーネル空間とのコンテキストスイッチが頻発し, そのオーバーヘッド自体が無視できない負荷となる. しかし,sched_ext では検証済みのバイトコードをカーネル内で JIT コンパイルして実行するため, 最小限のオーバーヘッドで高度なスケジューリングロジックを実現できる. 実験結果が示す通り,eBPF による制御は実用十分な速度で動作しており, アプリケーションの実行用リソースを最大限確保しつつ省電力性が求められるエッジ環境において,sched_ext は極めて合理的な選択肢であると結論付けられる.

6.6 既存の冷却手法との比較

既存の熱対策として標準的な DVFS は,CPU 温度が閾値を超えた際にクロック周波数を強制的に低下させる. これは確実な冷却効果をもたらすが, システム上で動作する全てのタスクが一律に遅くなるという欠点を持つ.

本研究の手法は, ハードウェアのクロックを下げるのではなく, ソフトウェア的にタスクの実行権を制御するアプローチをとった. これにより,DVFS では不可能な特定のタスクのみ高速に実行し続け, それ以外を減速させて冷却するという選択的な制御を実現した. これは, リアルタイム性が求められる制御タスクと, バッチ処理的な解析タスクが混在するエッジ AI などの用途において,DVFS よりも優れた特性を発揮すると考えられる.

提案手法では TOP-IL ほどの温度抑制を行うことは叶わなかった. ただし,TOP-IL は専用のプロセッサーを必要としているうえにそのようなものは温度抑制ではなく, ユーザー側のアプリケーションなどで使用することが前提となっている.

そのため, リソースを占有することなく温度抑制が実現できたのは本システムの実用面での有効性を示したと言える.

6.7　高負荷・飽和状態における限界と課題

温度が閾値付近で頻繁に上下する挙動や, ヒステリシス制御が十分に機能しない場面が観測された. これは, 現在の実装における温度ステージの切り替え判定が, 静的な閾値に依存していることに起因すると考えられる.

負荷が変動し続ける実環境では, 熱慣性によって制御の反映に時間差が生じるため, 単純な閾値制御では偏った結果になりやすい. より安定した制御を実現するためには,PID 制御のように温度の変化率を考慮に入れた動的な実行比率の調整や, 過去の負荷傾向を学習してステージ遷移を行う仕組みの導入が今後の課題となる.

第 7 章

おわりに

本研究は優先したいタスクの応答性と温度抑制の両立を実現することを目的として高温時に優先ではないタスクの実行頻度を減らすことで CPU の負担を軽減するシステムを考案した.

7.1 節では研究全体の総括を,7.2 節では得ることができた知見を,7.3 節では課題点を,7.4 節では今後の展望について述べる.

7.1 研究全体の総括

本研究では,Raspberry Pi 5 をはじめとする冷却性能に制約のあるエッジデバイスにおいて, タスクスケジューリングによるソフトウェア的なアプローチで熱問題を解決するシステムを提案した.Linux の最新機能である sched_ext を活用し, タスクの重要度と CPU 温度に基づいた動的スケジューラを設計・実装した.

実験による評価の結果, 提案手法は高負荷時において平均温度を約 6 ℃低下させつつ, 優先タスクの応答性を維持することに成功した. これにより, ハードウェア的な冷却強化が困難な環境下でも, ソフトウェアの工夫によってシステムの信頼性と重要タスクのリアルタイム性を両立できることを示した.

7.2 本研究で得られた知見

本研究を通じて, タスクの実行頻度制御が発熱抑制に対して有効な手段であることが確認された. 特に, 優先タスクの割合を全体の 20% 以下に抑えることで, システム全体が飽和するような状況下でも優先制御が機能するという定量的な知見が得られた. また,eBPF を用いたユーザ空間スケジューリングは, オーバーヘッドを許容範囲内に収めつつ, 柔軟な熱管理を実現できる実用的な技術であることが実証された.

7.3 本研究の課題

本研究では温度に基づき重要なタスクの応答性を維持しつつ温度抑制をするシステムを提案し, 一定の有効性を示した. しかしながら, 本研究にはいくつか課題が存在する.

第 1 に優先タスクとそうでないタスクの判別が事実上手動による登録に依存しきっている点である. Linux カーネルにおいてプロセスの状態を管理するための task_struct 構造体のように, タスクの詳細が保存されている構造体はいくつか存在するため, それらを駆使すれば今見ているタスクが優先されるべきものかどうかを判別することは可能である. しかしながらそれらを全てのタスクで逐一確認, 判断するとなるとカーネル空間での判別であってもオーバーヘッドが無視できないものとなってくる. 優先タスクは全体の 1$\tilde{2}$ 割であるとバランスが取れるということを示したが, 負荷の種類が違ってもその比率を維持する方法を見るけることができなかった.

第 2 に閾値が静的であることである. 温度やカスタム DSQ に溜まっているタスクの数を基に制限をかけているが, 閾値が静的であるため瞬間的な負荷と長時間の負荷で若干挙動が変わってしまう. 特に, 高温時かつ優先

タスクが殆ど無い場合は実質空回りをしてしまうことが多発する. アイドル状態を入れるために比率を変えているが, それは一定の割合で優先タスクが発生しているのが前提である. 優先タスクの割合が極端に少ない場合はその他タスクも例外的に優先タスクに登録するなどの処置を導入すればもう少し実行時間が早くなったものと思われる.

7.4 今後の展望

今後の課題として, 温度制御アルゴリズムの高度化が挙げられる. 現在の段階的なステージ制御に加え, 温度変化率を考慮した連続的な制御を取り入れることで, より滑らかな温度推移と安定した性能維持が可能になると考えられる. また, 本研究では CPU 負荷のみを対象としたが,GPU やメモリ帯域の制御も組み合わせることで, より包括的な熱エネルギー管理システムへと発展できる可能性がある.

謝辞

本研究の遂行にあたり, 宮澤准教授には研究構想から論文完成に至るまで, 終始熱心なご指導を賜りました. 特に, 議論が行き詰まった際に頂いた的確なご助言は, 本研究を完遂する大きな支えとなりました. 深く感謝 いたします. また, 切磋琢磨し合った宮澤研究室の皆様, 諸氏に厚く御礼申し上げます. 最後に, 大学生活を支え, 温かく見守ってくれた家族, 支えてくださった全ての方々に心より感謝の意を表 します.

参考文献

[1] M.Rapp, et.al: “NPU-Accelerated Imitation Learning for Thermal Optimization of QoS-Constrained Heterogeneous Multi- Core” *2024 ACM Transactions on Design Automation of Electronic Systems, Vol.29 No1, Article 16.*

[2] eBPF“eBPF Documentation” https://ebpf.io/what-is-ebpf/ (accessed September 29.)

[3] The Linux Kernel “Extensible Scheduler Class” https://docs.kernel.org/scheduler/sched-ext.html (accessed September 29.)